



ERASMO: Leveraging Large Language Models for Enhanced Clustering Segmentation

Fillipe dos Santos Silva^{1,2,3(✉)}, Gabriel Kenzo Kakimoto^{1,2,4}, Julio Cesar dos Reis^{1,3}, and Marcelo S. Reis^{1,2,3}

¹ Hub de Inteligência Artificial e Arquiteturas Cognitivas (H.IAAC),
Campinas, Brazil

{fillipesantos,g234878,jreis,msreis}@ic.unicamp.br

² Artificial Intelligence Laboratory (Recod.ai), Campinas, Brazil

³ Instituto de Computação, Universidade Estadual de Campinas (UNICAMP),
Campinas, Brazil

⁴ Faculdade de Engenharia Mecânica, Universidade Estadual de Campinas
(UNICAMP), Campinas, Brazil

Abstract. Cluster analysis plays a crucial role in various domains and applications, such as customer segmentation in marketing. These contexts often involve multimodal data, including both tabular and textual datasets, making it challenging to represent hidden patterns for obtaining meaningful clusters. This study introduces ERASMO, a framework designed to fine-tune a pretrained language model on textually encoded tabular data and generate embeddings from the fine-tuned model. ERASMO employs a textual converter to transform tabular data into a textual format, enabling the language model to process and understand the data more effectively. Additionally, ERASMO produces contextually rich and structurally representative embeddings through techniques such as random feature sequence shuffling and number verbalization. Extensive experimental evaluations were conducted using multiple datasets and baseline approaches. Our results demonstrate that ERASMO fully leverages the specific context of each tabular dataset, leading to more precise and nuanced embeddings for accurate clustering. This approach enhances clustering performance by capturing complex relationship patterns within diverse tabular data.

Keywords: Clustering Segmentation · Transformer-based Models · Tabular Data Embeddings

1 Introduction

Tabular data is ubiquitous in various fields such as finance, healthcare, and marketing, where it serves as a primary source of information for Machine Learning

(ML) tasks [15]. Despite its widespread use, extracting meaningful insights from tabular data remains a complex challenge, particularly in clustering tasks [6]. These challenges include handling heterogeneous feature types, dealing with high-dimensional spaces, and ensuring meaningful distance metrics. These can significantly impact the effectiveness of clustering algorithms in identifying natural groupings within the data [14].

Recent researchers have explored traditional statistical methods and modern deep learning approaches [14, 19, 21] to address these challenges. Existing studies have utilized Large Language Models (LLMs) like OpenAI’s GPT and LLaMA to create embeddings from textual datasets, enhancing data representation and analysis [6, 14]. In addition, a method combining LLMs and Deterministic, Independent-of-Corpus Embeddings (DICE) has been proposed to generate consistent embeddings across datasets, improving segmentation accuracy [19]. However, these approaches often fail to fully leverage the specific context of each tabular dataset, resulting in less nuanced embeddings for precise clustering.

This study originally introduces ERASMO, our proposed framework designed to generate high-quality embeddings from tabular data using transformer-based language models. These embeddings excel in clustering analysis, revealing hidden patterns and groupings. Our solution can also be used in Retrieval-Augmented Generation (RAG) systems and other tasks to gain deeper insights from context information [7]. ERASMO operates through two stages: (1) fine-tuning a pretrained language model on textually encoded tabular data; and (2) generating embeddings from the fine-tuned model. Using techniques like random feature sequence shuffling and number verbalization, ERASMO produces contextually rich and structurally representative embeddings, outperforming all clustering strategies from the literature based on internal metrics.

Our experimental evaluation used three clustering quality metrics to compare ERASMO with state-of-the-art methods: Silhouette Score (SS), Calinski-Harabasz Index (CHI), and Davies-Bouldin Index (DBI). These metrics comprehensively assess clustering effectiveness results by measuring cohesion, separation, and overall cluster structure.

We extensively evaluated ERASMO on real-world datasets without true labels, including Banking Marketing Targets, E-Commerce Public Dataset by Olist, Yelp reviews, PetFinder.my, and Women’s Clothing Reviews. These datasets encompass diversified information and present various challenges, rigorously testing ERASMO’s clustering capabilities.

This article provides the main contributions as follows:

- We introduce ERASMO, a novel framework that leverages transformer-based language models to generate high-quality embeddings from tabular data, enhancing clustering analysis.
- We demonstrate that our framework significantly improves clustering performance by capturing the complex relationships within tabular data through random feature sequence shuffling and number verbalization techniques.

- We experimentally achieve state-of-the-art clustering results with ERASMO, showcasing its effectiveness in identifying patterns and groupings within diverse tabular datasets.
- To the best of our knowledge, ERASMO is the first framework to fully integrate and fine-tune transformer-based language models specifically for generating embeddings from tabular data, resulting in superior clustering outcomes.

The remainder of this article is organized as follows. Section 2 presents a synthesis of related work. Section 3 details the ERASMO framework. Section 4 outlines our experimental methodology. Section 5 presents the results of our evaluations. Section 6 discusses the implications of our findings. Finally, Sect. 7 provides conclusions and directions for future work.

2 Related Work

Several studies have explored the application of LLMs to transform tabular data for clustering tasks, demonstrating the potential to enhance user segmentation and data analysis [6, 14, 19, 21, 23, 24]. Zhu *et al.* [24] proposed a novel method named Word Embedding of Dimensionality Reduction (WERD) for document clustering. Their approach integrates pre-trained word embeddings with dimensionality reduction techniques. In their work, Sentence-BERT embeds them into high-dimensional vectors after preprocessing documents, which PaCMAP then reduces. Spectral clustering is applied, followed by Non-Negative Matrix Factorization to extract keywords.

CLUSTERLLM [23], a novel text clustering framework, leverages feedback from LLMs such as ChatGPT. This method enhances clustering by utilizing LLMs to refine clustering perspectives and granularity through two stages: a triplet task for fine-tuning embedders based on user preferences and a pairwise task for determining cluster granularity. Extensive experiments on fourteen datasets demonstrated that CLUSTERLLM consistently improves clustering quality and is cost-effective, outperforming traditional clustering methods. Both WERD [24] and CLUSTERLLM [23] presented limitations compared to the ERASMO framework (our proposal). WERD might not fully capture the contextual nuances of each dataset due to its focus on dimensionality reduction techniques. At the same time, CLUSTERLLM’s reliance on general-purpose LLMs for guidance may overlook specific dataset characteristics.

A method demonstrating that LLMs enables few-shot learning applied to clustering tasks was proposed in [21]. Their study showed how LLMs can perform clustering tasks with minimal labeled data by leveraging their extensive pretraining, significantly reducing the need for large annotated datasets and achieving reasonable clustering performance with few-shot learning. Similarly, Tipirneni *et al.* [18] explored context-aware clustering using LLMs, highlighting how these models can utilize contextual information to enhance clustering accuracy. Both methods, however, may not fully leverage the dataset-specific nuances as effectively as ERASMO because we employ a fine-tuning step, allowing the model to

capture better and utilize dataset-specific details, leading to more accurate and reliable clustering results.

Tissera, Asanka, & Rajapakse developed [19] an approach to enhancing customer segmentation using LLMs and DICE. Their method combined LLMs with DICE to generate consistent and deterministic embeddings across different datasets, improving segmentation accuracy and robustness. Their approach may not fully leverage the context-specific nuances of each dataset as effectively as ERASMO. Our fine-tuning process proposal allows it to adapt to the unique characteristics of the input data, providing more contextually rich and detailed embeddings that can result in more precise and meaningful clusters.

A comparative analysis of LLM embeddings for effective clustering was explored in [6]. As an extension, the study on text clustering with LLM embeddings [14] delves deeper, exploring additional models and datasets to demonstrate improvements in text data clustering. While existing approaches effectively capture complex semantic relationships and handle categorical, numerical, and textual data, they lack the fine-tuning specificity of ERASMO, our key originality aspect. ERASMO’s tailored embeddings for tabular datasets and integration of feature order permutation provide more precise and contextually relevant clusters, offering superior versatility and robustness in various clustering applications.

3 ERASMO

This section introduces ERASMO, our framework that leverages transformer-based language models to generate high-quality embeddings from tabular data. These embeddings are particularly effective for clustering analysis, allowing for identifying patterns and groupings within the data that might not be immediately apparent.

The process involves two main stages: (1) fine-tuning a pretrained LLM on a textually encoded tabular dataset; and (2) utilizing the fine-tuned model to generate embeddings, which are used by a clustering algorithm. These designed stages were inspired by [2]. Subsection 3.1 details the fine-tuning phase, whereas Subsect. 3.2 reports on the embedding generation processes.

3.1 Phase 1: Fine-Tuning

Standard pretrained generative LLMs expect sequences of words as inputs. Hence, we convert each row of our dataset into a textual representation to apply an LLM to tabular data, which can contain categorical, numerical, and textual information.

Definition 1 (Textual Converter). *Given a tabular dataset with m columns with feature names $f_1, f_2, \dots, f_m$ and n rows of samples $s_1, \dots, s_n$, let the entry $v_{i,j}$, $i \in \{1, \dots, n\}$, $j \in \{1, \dots, m\}$ represent the value of the j -th feature of the i -th data point. Taking the feature name and value into account, each sample s_i*

of the table is transformed into a textual representation t_i using the following subject-predicate-object transformation:

$$t_{i,j} = [f_j, \text{"is"}, v_{i,j}, \text{"}, \text{"}], \quad \forall i \in \{1, \dots, n\}, j \in \{1, \dots, m\} \quad (1a)$$

$$t_i = [t_{i,1}, t_{i,2}, \dots, t_{i,m}], \quad \forall i \in \{1, \dots, n\}, \quad (1b)$$

where $t_{i,j}$, the textually encoded feature, is a clause with information about a single value and its corresponding feature name, and $[\cdot]$ denotes the concatenation operator.

By transforming a tabular feature vector into a sequence using the textual subject-predicate-object encoding scheme, pseudo-positional information is artificially introduced into the transformed tabular data sample. However, there is no spatial ordering relationship between features in tabular datasets. We randomly permute the encoded short sentences $t_{i,j}$ of the full textual representation t_i to reconstruct the feature order independence.

Definition 2 (Random Feature Sequence Shuffle). Let $t_i, i \in \{1, \dots, n\}$, be a textual representation. Consider a sequence $k = (k_1, \dots, k_m)$ that is a permutation of the sequence of indices $(1, \dots, m)$. A random feature sequence shuffle is defined as $t_i(k) = [t_{i,k_1}, t_{i,k_2}, \dots, t_{i,k_m}]$.

We fine-tune our generative language model on samples without order dependencies when using shuffled orders of the textually encoded features. Moreover, such permutations are highly beneficial as they allow for arbitrary conditioning in tabular data generation. In our experiments, we refer to ERASMO_{base} as the baseline model, utilizing only the Textual Converter and Random Feature Sequence Shuffle. In addition, there is evidence that verbalizing numerical tokens can enhance effectiveness in specific scenarios [8]. In this sense, we explore this approach, naming it ERASMO_{NV}, as follows.

Definition 3 (Number Verbalizer). Let $t_i, i \in \{1, \dots, n\}$, be a textual representation, and $t_{i,j}$ be the set of words of the j -th feature of t_i . A number verbalizer is a function v that receives as input a word w of $t_{i,j}$ and is defined as:

$$v(w) = \begin{cases} w, & \text{if } w \text{ is not numerical,} \\ \text{verbalized } w & \text{otherwise.} \end{cases} \quad (2)$$

By applying this transformation on every token of every textual representation, we ensure that any numerical information in the text is verbalized. In some NLP tasks, such as clustering with embeddings, sentiment analysis, and text classification, verbalizing numbers can improve the model’s understanding of the context and meaning of numerical values, leading to more accurate and meaningful results. This transformation might not be beneficial in some cases, depending on the specific nature of the data and the task at hand [8,9].

Fine-Tuning a Pretrained Auto-Regressive Language Model : We describe the fine-tuning procedure of a pretrained LLM on the encoded tabular data for generation tasks. We suppose a textually encoded tabular dataset $T = \{t_i(k)\}_{i=1,\dots,n}$ that was transformed into text by the proposed encoding scheme. Let k be a randomly drawn permutation, and n denote the number of rows. Based on user choice, the pipeline can proceed directly to fine-tuning the LLM to generate ERASMO_{base} , or it can first apply a number verbalizer to convert numerical tokens into their verbal representations before fine-tuning the LLM to generate ERASMO_{NV} .

To be processed with an LLM, the input sentences $t \in T$ must be encoded into a sequence of tokens from a discrete and finite vocabulary W . These tokens can be character, word, or subword encodings such as the Byte-Pair-Encodings (BPE). Thus, $t \in T$ is represented by a sequence of tokens $(w_1, \dots, w_j) = \text{TOKENIZE}(t)$ with tokens $w_1, \dots, w_j \in W$, where j denotes the number of tokens required to describe the character sequence t . Commonly, the probability of natural-language sequences is factorized in an auto-regressive manner in LLMs. It is represented as a product of output probabilities conditioned on previously observed tokens:

$$p(t) = p(w_1, \dots, w_j) = \prod_{k=1}^j p(w_k \mid w_1, \dots, w_{k-1}). \quad (3)$$

As a result, an end-user can choose any existing generative language model for tabular data modeling and exploit the vast amount of contextual knowledge presented in these models. Fine-tuning enables the model to leverage this contextual information with the feature and category names to enhance the model’s capabilities. Figure 1 presents the pipeline for ERASMO’s fine-tuning step.

3.2 Phase 2: Embedding Generation and Clustering Analysis

We generate embeddings from the model after fine-tuning the LLM on the textually encoded tabular dataset. These embeddings capture the contextual relationships and features encoded during the training phase.

We start by feeding the test dataset, transformed into its textual representation, into the fine-tuned LLM. The model generates embeddings for each input sequence, providing a high-dimensional representation for each sample. This process ensures that the embeddings preserve the contextual and feature relationships learned during fine-tuning. Depending on the user’s choice in the pipeline, the embeddings are generated from either ERASMO_{base} or ERASMO_{NV} models, reflecting whether the number verbalizer step was applied.

To generate these embeddings, the input sentences $t \in T_{test}$ are encoded into sequences of tokens and processed by the fine-tuned LLM. The embeddings are obtained from the final hidden states of the model, resulting in rich and informative representations of the data. These embeddings can then be utilized for various downstream tasks, including clustering analysis, to gain deeper insights into the data structure (cf. Figure 2).

4 Experimental Methodology

Our experiments evaluated the quality assessment for the best-performing clustering algorithms for each dataset and approach (model) combination (cf. Table 1 for the obtained results). Subsection 4.1 describes the datasets used for training and testing. Subsection 4.2 presents an overview of the clustering algorithms. Subsection 4.3 describes the approaches used for comparison as baselines in our experiments. Subsection 4.4 reports on the evaluation metrics. Subsection 4.5 presents the implementation details. Each experimental setup for a given dataset assessed the different language models fine-tuned and pretrained. Each model considers the several clustering algorithms and their configuration.

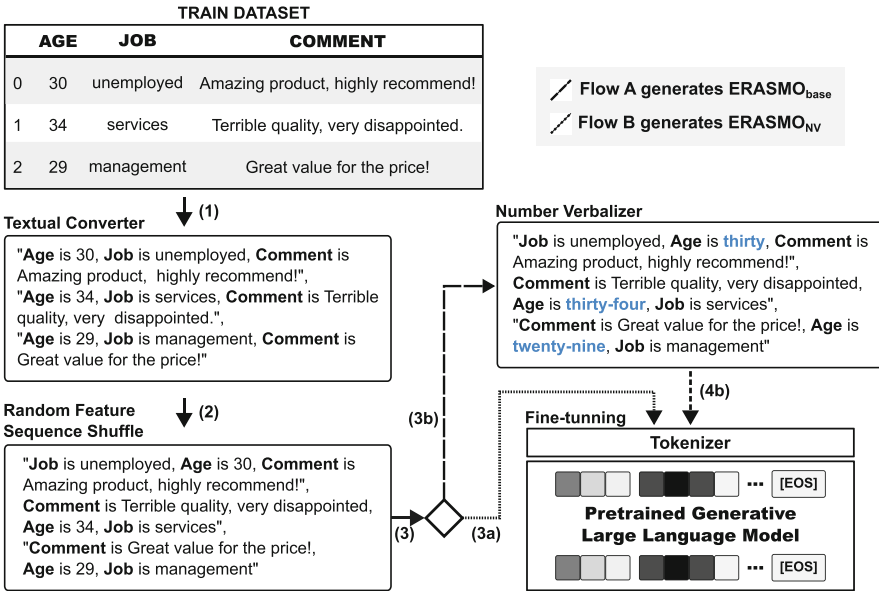


Fig. 1. The ERASMO data pipeline for the fine-tuning phase. First, a textual converter step transforms tabular data into meaningful text (1). Next, a random feature order permutation step is applied (2). Then, based on user choice, the pipeline diverges: it can proceed directly to fine-tuning a LLM (3a) to generate ERASMO_{base}, or apply a number verbalizer (3b) before fine-tuning the LLM (4b) to generate ERASMO_{NV}.

4.1 Datasets

We selected a diversified set of datasets to encompass a variety of challenges related to text categorization and clustering, and we used them to evaluate text clustering algorithms.

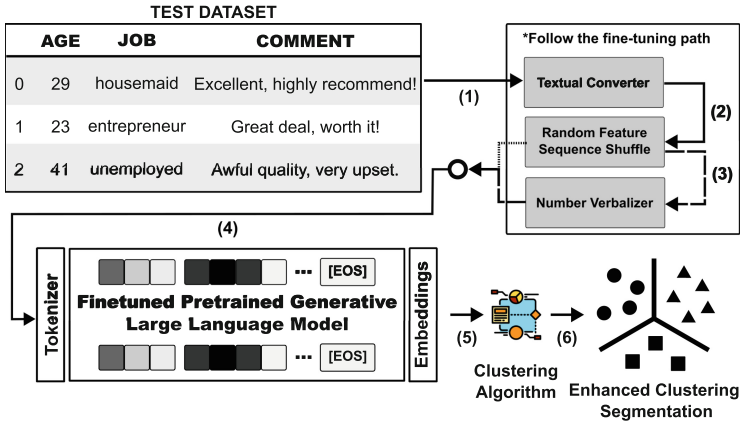


Fig. 2. The ERASMO pipeline for generating embeddings and cluster analysis. The input test tabular data is first transformed into text sequences (1). Next, a random feature order permutation step is applied (2). For ERASMO_{NV}, a number verbalizer step follows (3) before processing by the fine-tuned LLM to generate embeddings (4). For ERASMO_{base}, the pipeline goes directly from step (2) to step (4). The embeddings are subsequently used for clustering analysis.

- **Banking Marketing Targets:** Composed of data from direct marketing campaigns of a banking institution, which includes client attributes like age and job, along with the response to the campaign [10].
- **E-Commerce Public Dataset by Olist:** A Brazilian e-commerce dataset with over 100,000 orders from 2016 to 2018 across multiple marketplaces [11]. It includes 72,794 training and 18,199 testing samples. The Recency, Frequency, Monetary (RFM) model was used for customer segmentation, as described in [19].
- **Yelp:** Comprises reviews from Yelp businesses, including text reviews, star ratings, and business attributes, offering a rich resource for sentiment analysis and review classification tasks [4].
- **PetFinder.my:** Features adoption records from the PetFinder.my website, encompassing various pet attributes, descriptions, and adoption status, valuable for text classification and clustering related to animal welfare [5].
- **Women Clothing Reviews:** Contains reviews of women’s clothing, with each review detailing text feedback, ratings, and customer information, suitable for sentiment analysis and recommendation system research [3].

Each unlabeled dataset was processed through the proposed pipeline, which involves training a pretrained LLM. This approach ensures that the clustering algorithms can perform optimally across diverse textual inputs, enhancing their ability to effectively identify and group related data points.

4.2 Clustering Algorithms

The clustering algorithms chosen are well-suited for handling complex patterns in structured and textual data, ensuring efficient categorization.

We used the k -means algorithm for its simplicity and effectiveness with large datasets and k -means++ for its strategic centroid initialization to enhance clustering efficiency and quality [12]. Unlike k -means, which assigns each data point to a single cluster, Fuzzy C-Means (FuzzyCM) employs a probabilistic membership approach, effectively capturing the nuances and polysemy typical of textual data. We used Agglomerative Hierarchical Clustering (AHC) to uncover hierarchical structures and spectral clustering for its proficiency in recognizing clusters based on the data’s graph structure, effectively identifying non-convex shapes.

For k -means, the parameters were: initialization method set to random, number of initializations (*ninit*) set to 10, and the random seed set to 0. The k -means++ algorithm utilized k -means++ for initialization, *ninit* was 1, and the seed was 0. Agglomerative Hierarchical Clustering (AHC) employed the Euclidean metric with Ward linkage. For Fuzzy C-means (FuzzyCM), no initialization method was specified, the fuzziness parameter (m) was 2, error tolerance was set to 0.005, and the maximum number of iterations (*maxiter*) was 1000. Spectral clustering used ‘discretize’ for label assignment and a random seed of 10.

Implementations for these algorithms were sourced from the *scikit-learn* library [13], except for FuzzyCM, which used the *scikit-fuzzy* package [22]. For k -means and k -means++, *init* specifies the initial cluster centroid method, *ninit* indicates the number of algorithm runs with different seeds, and *seed* sets the random number for centroid initialization. In AHC, *metric* is the metric used for linkage computation, and *linkage* is the criterion measuring the distance between observation sets. We used Euclidean distance to measure point similarities and a nearest centroid approach to associate clusters.

For FuzzyCM, *init* is the initial fuzzy c-partitioned matrix (random if *None*), m is the fuzziness degree, *error* is the stopping criterion, and *maxiter* is the iteration limit. In Spectral clustering, *assign_labels* specifies the labeling strategy in the embedding space, and *seed* is the pseudorandom number for initializing the eigenvector decomposition. For all datasets, the number of clusters (k) was determined using the silhouette score to optimize cluster cohesion and separation.

4.3 Approaches (Baselines)

We utilized various embedding techniques from state-of-the-art LLMs, including OpenAI, Falcon, Llama 2, GPT-2 Medium, and an MPNet-based model, each enhancing text representation by capturing contextual nuances.

For the MPNet-based model, we used **sentence-transformers/all-mpnet-base-v2** (MPNet-v2) [17]. For the OpenAI model, we utilized **text-embedding-3-large**, and for the Falcon model, we used **tiiuae/falcon-7b** [1]. Additionally, we employed **Llama-2-7b-chat-hf** [20] for chat applications and

gpt2-medium [16] for faster textual representations. We used the embeddings from the last layer of all models for the most contextually rich text representations.

We integrated an additional baseline from a recent study that explores customer segmentation using LLMs combined with DICE [19]. They used the **paraphrase-multilingual-mpnet-base-v2** model from Sentence Transformers to generate 768-dimensional sentence embeddings. This model, based on the MPNet architecture, has about 278 million parameters and is designed for clustering and semantic search.

4.4 Evaluation Metrics

We use a set of metrics to evaluate our proposed framework and baselines thoroughly. Specifically, we employed the SS, CHI, and DBI metrics to assess the cohesion, compactness, and separation of clusters, ensuring a robust analysis of their structural integrity.

The SS metric, which assesses the separation and cohesion of clusters, is calculated for each data point i as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}},$$

where $a(i)$ measures the average intra-cluster distance, and $b(i)$ is the minimum inter-cluster distance for the point i .

CHI measures the ratio of between-cluster dispersion to within-cluster dispersion, providing insights into the overall clustering structure. The index is formulated as:

$$CHI = \frac{\text{Tr}(B_k)/(k-1)}{\text{Tr}(W_k)/(N-k)},$$

where $\text{Tr}(B_k)$ is the trace of the between-group dispersion matrix, and $\text{Tr}(W_k)$ the trace of the within-group dispersion matrix, thus evaluating both the separation and compactness of the clusters.

The DBI evaluates the average similarity ratio of each cluster with its most similar one, offering a measure of cluster separation. The index is calculated as:

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d_{ij}} \right),$$

where σ_i and σ_j represent the average distance of all elements in clusters i and j to their respective centroids, and d_{ij} is the distance between centroids of clusters i and j . Lower values of DBI indicate better cluster separation.

4.5 Our Implemented Setup

We compare the described baselines (cf. Subsection 4.3) using a pretrained transformer-decoder LLM model, *GPT-2 medium* [16], which has 355 million trainable parameters, 24 layers, 16 attention heads, an embedding size of 1024, and a context size of 1024. To facilitate this comparison, we convert the tabular

dataset into text for all baselines, applying the random feature sequence shuffling function and comparing the results with ERASMO_{base} and ERASMO_{NV} . Both models were trained with a batch size of 8 over 60 epochs. We applied a dropout rate of 0.1 and utilized 500 warmup steps. The models incorporated a weight decay of 0.01 and used the Adam optimizer with ϵ set to $1e-8$ and β values of $[0.7, 0.9]$. The initial learning rate was set to $5e-5$, with a schedule starting at $1e-8$, ranging from a minimum of $1e-5$ to a maximum of $4e-5$. The model was developed using PyTorch¹ and is made available at a GitHub repository². It ran on a system equipped with five NVIDIA RTX A6000, each having 48 GB of Random Access Memory (RAM).

5 Experimental Results

We present key results obtained organized by datasets. Table 1 presents the SS, CHI, and DBI metrics results for the test dataset for all evaluated approaches; values in bold indicate the best outcomes. The best algorithm was determined by choosing the algorithm with the highest SS.

Banking. In the Banking dataset, the ERASMO_{base} strategy outperformed all other strategies, achieving the highest SS of 0.75, the highest CHI of 12,038.44, and the lowest DBI of 0.37. This indicates well-defined and compact clusters. ERASMO_{NV} also performed strongly with an SS of 0.71 and CHI of 7,570.38, though it had a slightly higher DBI of 0.43 compared to ERASMO_{base} . Other strategies like MPNet-v2 and Falcon showed moderate performance with SS values of 0.27 and 0.23, respectively, while OpenAI had the lowest SS at 0.11 and the highest DBI at 2.73, indicating poor clustering.

Olist. For the Olist dataset, ERASMO_{NV} achieved the highest SS of 0.77, indicating the best clustering quality. It reached the highest CHI of 62,036.87 and a low DBI of 0.32. ERASMO_{base} followed closely with an SS of 0.75 and a CHI of 54236.31, along with the lowest DBI of 0.30. Other strategies like LLaMA-2 and Falcon performed reasonably well, with SS values of 0.71 and 0.66, respectively. However, OpenAI and MPNet-v2 showed lower SS values, with OpenAI achieving an SS of 0.19 and MPNet-v2 an SS of 0.24.

Yelp. In the Yelp dataset, ERASMO_{NV} and ERASMO_{base} both demonstrated superior performance, with SS values of 0.79 and 0.78, respectively. ERASMO_{NV} also achieved the highest CHI of 8,410.94 and tied with ERASMO_{base} for the lowest DBI of 0.28. Other strategies, such as GPT2 Medium and LLaMA-2, showed moderate clustering performance with SS values of 0.39 and 0.29, respectively. OpenAI, with an SS of 0.07, and MPNet-v2, with an SS of 0.23, indicated less effective clustering.

¹ pytorch.org.

² [ERASMO - GitHub](#).

Table 1. This table shows the clustering quality assessment results for the top-performing algorithms across each dataset and approach combination. The best algorithm was chosen based on the highest SS. We provide the optimal number of clusters (k) and the results for SS, CHI, and DBI. Bold values indicate the best results for each metric.

Dataset	Approach	Best alg.	Best k	SS	CHI	DBI
Banking	MPNet-v2	k -means	2	0.27	1,981.53	1.46
	OpenAI	k -means	9	0.11	212.96	2.73
	LLaMA-2	k -means++	8	0.22	593.66	1.66
	Falcon	k -means	2	0.23	1,776.67	1.56
	GPT2 Medium	k -means	2	0.40	4,764.08	0.90
	PMV2 + DICE	k -means	2	0.31	2,389.25	1.33
	ERASMO _{base}	k -means	2	0.75	12,038.44	0.37
	ERASMO _{NV}	AHC	2	0.71	7,570.38	0.43
Olist	MPNet-v2	k -means	2	0.24	5,927.77	1.59
	OpenAI	k -means	3	0.19	3,946.14	1.83
	LLaMA-2	k -means	4	0.71	43,306.34	0.45
	Falcon	k -means	6	0.66	45,512.63	0.55
	GPT2 Medium	k -means	2	0.48	26,471.54	0.75
	PMV2 + DICE	SC	2	0.61	27,578.16	0.67
	ERASMO _{base}	SC	2	0.75	54,236.31	0.30
	ERASMO _{NV}	k -means	2	0.77	62,036.87	0.32
Yelp	MPNet-v2	AHC	2	0.23	36.61	2.25
	OpenAI	AHC	2	0.07	38.78	3.86
	LLaMA-2	k -means	10	0.29	445.87	1.42
	Falcon	k -means++	14	0.32	442.83	1.22
	GPT2 Medium	k -means	2	0.39	1,898.25	1.00
	PMV2 + DICE	AHC	2	0.53	32.86	1.08
	ERASMO _{base}	SC	2	0.78	7,702.89	0.28
	ERASMO _{NV}	AHC	2	0.79	8,410.94	0.28
PetFinder.my	MPNet-v2	k -means	2	0.14	236.58	2.28
	OpenAI	AHC	2	0.16	3.29	1.75
	LLaMA-2	k -means++	17	0.35	179.00	1.38
	Falcon	k -means++	2	0.20	397.65	1.86
	GPT2 Medium	AHC	2	0.55	636.2	0.70
	PMV2 + DICE	k -means	5	0.18	242.83	1.85
	ERASMO _{base}	k -means	2	0.72	3,351.95	0.40
	ERASMO _{NV}	AHC	2	0.73	3,063.55	0.34
Clothings	MPNet-v2	k -means	3	0.12	195.01	2.47
	OpenAI	SC	5	0.07	70.17	2.90
	LLaMA-2	k -means++	9	0.37	435.45	1.53
	Falcon	k -means	12	0.24	326.64	1.54
	GPT2 Medium	k -means	2	0.52	3,541.87	0.68
	PMV2 + DICE	AHC	2	0.17	72.74	1.97
	ERASMO _{base}	k -means	2	0.72	6,208.52	0.39
	ERASMO _{NV}	k -means++	2	0.71	5,916.57	0.39

PetFinder.my. In the PetFinder.my dataset, ERASMO_{NV} slightly outperformed ERASMO_{base} with an SS of 0.73 compared to 0.72. ERASMO_{base} had the highest CHI of 3,351.95 and a low DBI of 0.40, while ERASMO_{NV} had a CHI of 3,063.55 and the lowest DBI of 0.34. Other strategies, such as GPT2 Medium and Falcon, showed moderate results with SS values of 0.55 and 0.20, respectively. MPNet-v2 had the lowest SS of 0.14, indicating poor clustering performance.

Clothings. For the Clothings dataset, ERASMO_{base} achieved the highest SS of 0.72 and the highest CHI of 6,208.52, along with the lowest DBI of 0.39. ERASMO_{NV} also performed well with an SS of 0.71 and a CHI of 5,916.57, matching the lowest DBI of 0.39. Other strategies like GPT2 Medium and LLaMA-2 showed moderate performance, with SS values of 0.52 and 0.37, respectively. OpenAI and MPNet-v2 had the lowest SS values of 0.07 and 0.12, respectively, indicating poor clustering quality.

Figure 3 shows a 2D t-Distributed Stochastic Neighbor Embedding (t-SNE) visualization for the Yelp dataset across all strategies. The Yelp dataset was chosen for its rich and diverse user reviews, making it ideal for clustering evaluation. Our models, ERASMO_{base} and ERASMO_{NV}, display distinct and well-separated clusters, reflecting their lowest DBI scores. This highlights the superior clustering performance of our models.

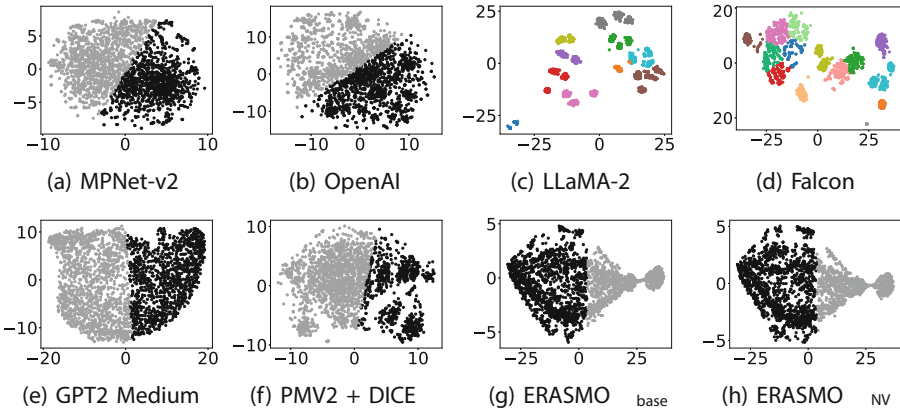


Fig. 3. t-SNE visualization of embedding representations on the Yelp dataset for different models: (a) MPNet-v2, (b) OpenAI, (c) LLaMA-2, (d) Falcon, (e) GPT2 Medium, (f) PMV2 + DICE, (g) ERASMO_{base}, and (h) ERASMO_{NV}.

6 Discussion

The SS results consistently highlight the superior clustering effectiveness of the ERASMO_{base} and ERASMO_{NV} strategies across all datasets. ERASMO_{base}

achieved the highest SS in the Banking and Clothing datasets, while ERASMO_{NV} led in the Olist, Yelp, and PetFinder.my datasets. This indicates that both strategies help to form well-defined clusters, with ERASMO_{base} having a slight edge in some datasets. The higher SS values for these models suggest they are better at creating distinct clusters than other strategies like MPNet-v2, OpenAI, and Falcon, which showed significantly lower SS values, indicating poorer representation relevance for clustering quality.

The DBI results further support the effectiveness of ERASMO_{base} and ERASMO_{NV}. Both strategies consistently achieved the lowest DBI values, particularly excelling in the Banking, Olist, and Yelp datasets. A lower DBI indicates more compact and well-separated clusters, affirming that our proposed models form tight and distinct clusters. ERASMO_{base} generally showed slightly better DBI scores, suggesting it produces more compact clusters than ERASMO_{NV}. In contrast, approaches like OpenAI and MPNet-v2 had significantly higher DBI values, reflecting less adequate cluster compactness and separation.

The CHI results corroborate the trends observed in SS and DBI. ERASMO_{base} and ERASMO_{NV} achieved the highest CHI scores across most datasets, with ERASMO_{base} leading in the Banking and Clothing datasets, and ERASMO_{NV} in the Olist and Yelp datasets. High CHI scores indicate that our proposed models create clusters that are not only well-separated but also highly dense. These superior results across multiple metrics underscores the robustness and effectiveness of the ERASMO approach. Other models like MPNet-v2, OpenAI, and Falcon demonstrated lower CHI scores, indicating less effective clustering effectiveness.

ERASMO has limitations, including reliance on high-quality data, challenges with ambiguous datasets, and significant computational demands for fine-tuning. Random sequence shuffling can affect reproducibility, and verbalizing numerical tokens does not consistently improve outcomes, warranting further investigation. Furthermore, applying traditional metrics such as SS, CHI, and DBI to analyze clusters from various representation spaces appears as an open research challenge yet to be resolved in the literature. While these metrics are designed for uniform representation spaces, they are frequently applied to different representation spaces, as evidenced by previous studies [6, 14]. We recognize these limitations and stress the relevance of investigating and developing further refined metrics capable of accurately assessing embeddings from varied dimensional spaces.

Despite these challenges, ERASMO significantly advances clustering techniques by enhancing data representation. This innovation sets the stage for the potential standardization of metrics and improves clustering quality. ERASMO stands as a crucial development in pursuing more robust clustering strategies.

7 Conclusion

Exploring structured and textual data simultaneously for clustering analysis is a challenging problem. This study presented and evaluated the ERASMO_{base} and ERASMO_{NV} clustering approaches, demonstrating their superior empirical

effectiveness across multiple datasets. The results, based on SS, DBI, and CHI, consistently revealed that our proposed models help to create well-defined, compact, and dense clusters, outperforming all strategies. Despite their high computational demands and dependency on data quality, we found the ERASMO models suited for practical clustering tools. Future work might focus on optimizing computational efficiency and enhancing robustness for diverse and noisy datasets.

Acknowledgements. This project was supported by the Brazilian Ministry of Science, Technology and Innovations, with resources from Law nº 8,248, of October 23, 1991, within the scope of PPI-SOFTEX, coordinated by Softex and published Arquitetura Cognitiva (Phase 3), DOU 01245.003479/2024 -10.

References

1. Almazrouei, E., et al.: Falcon-40b: an open large language model with state-of-the-art performance (2023)
2. Borisov, V., Seßler, K., Leemann, T., Pawelczyk, M., Kasneci, G.: Language models are realistic tabular data generators. arXiv preprint [arXiv:2210.06280](https://arxiv.org/abs/2210.06280) (2022)
3. Brooks, N.: Women's e-commerce clothing reviews (2017). <https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews>
4. Dataset, Y.: Yelp dataset (2014). http://www.yelp.com/dataset_challenge
5. Kaggle, PetFinder.my: Petfinder.my adoption prediction (2019). <https://www.kaggle.com/c/petfinder-adoption-prediction/data/>
6. Keraghel, I., et al.: Beyond words: a comparative analysis of llm embeddings for effective clustering. In: Int. Symposium on Intelligent Data Analysis, pp. 205–216. Springer, Heidelberg (2024)
7. Lewis, P., Perez, A., Petroni, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. In: NIPS (2020)
8. Liu, Y., He, X., Zhong, T., et al.: understanding llms: a comprehensive overview from training to inference. arXiv preprint [arXiv:2401.02038](https://arxiv.org/abs/2401.02038) (2024)
9. Min, B., Ross, D., others.: Recent advances in natural language processing via large pre-trained language models: a survey. ACM Comput. Surv. 1–40 (2023)
10. Moro, S. Rita, P., Cortez, P.: Bank marketing. UCI Machine Learning Repository (2012). <https://doi.org/10.24432/C5K306>
11. Olist: Brazilian e-commerce public dataset by olist (2023). <https://doi.org/10.34740/KAGGLE/DSV/195341>. acessado em: 03 de outubro de 2023
12. Ourabah, O., et al.: Large scale data using k-means. Mesopotamian J. Big Data **2023**, 36–45 (2023)
13. Pedregosa, F., Varoquaux, M., Prettenhofer, V., et al.: Scikit-learn: machine learning in python. J. Mach. Learn. Res. **12**, 2825–2830 (2011)
14. Petukhova, A., Matos-Carvalho, J.P., Fachada, N.: Text clustering with llm embeddings. arXiv preprint [arXiv:2403.15112](https://arxiv.org/abs/2403.15112) (2024)
15. Pitafi, S., Anwar, T., Sharif, Z.: A taxonomy of machine learning clustering algorithms, challenges, and future realms. Appl. Sci. **13**(6), 3529 (2023)
16. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al.: Language models are unsupervised multitask learners. OpenAI blog **1**(8), 9 (2019)
17. Song, K., et al.: MpNet: masked and permuted pre-training for language understanding. Adv. Neural Inf. Process. Syst. **33**, 16857–16867 (2020)

18. Tipirneni, S., Adkathimar, N., Hiranandani, V.N., Yuan, C., Reddy, C.K.: Context-aware clustering using large language models. arXiv (2024)
19. Tissera, M., Asanka, P., et al.: Enhancing customer segmentation using llms and deterministic, independent-of-corpus embeddings. In: 2024 4th (ICARC) (2024)
20. Touvron, H., Martin, L., Stone, K., et al.: Llama 2: open foundation and fine-tuned chat models. arXiv preprint [arXiv:2307.09288](https://arxiv.org/abs/2307.09288) (2023)
21. Viswanathan, V., Gashteovski, K., Lawrence, C., Wu, T., Neubig, G.: Large language models enable few-shot clustering. arXiv preprint [arXiv:2307.00524](https://arxiv.org/abs/2307.00524) (2023)
22. Warner, J., Sexauer, J., et al.: JDWarner/scikit-fuzzy: Scikit-Fuzzy version 0.4.2. Zenodo (2019). <https://doi.org/10.5281/zenodo.3541386>
23. Zhang, Y., Wang, Z., Shang, J.: Clusterllm: large language models as a guide for text clustering. arXiv preprint [arXiv:2305.14871](https://arxiv.org/abs/2305.14871) (2023)
24. Zhu, P., Lang, Q., Liu, X.: Word embedding of dimensionality reduction for document clustering. In: 2023 35th Chinese Control and Decision Conference (CCDC), pp. 4371–4376. IEEE (2023)